

Adaptive Model Compression for Efficient Image Classification

Perfect Makuwerere
Department of Computer Science
Stevens Institute of Technology
Email: pmakuwer@stevens.edu

Krishna S. Deb
Department of Computer Science
Stevens Institute of Technology
Email: kdeb@stevens.edu

I. INTRODUCTION

Artificial Intelligence (AI) models often for large applications require lot of computational resources like time. Memory requirements are often as large as giga bytes, which is often similar to or less than the total memory of small devices like mobile phones and others [1] [2]. Several IOT devices like smart watches with lots of sensors, face recognizing door locks etc., are having very small processing powers [3].

It needs to be sustainable for environment as well. "It is estimated that the GBT-3 network, with 175 Billion parameters, emitted a total of 552.1 tons of carbon dioxide in training." [4].

Thus more efficient networks are required which will be fulfilling our requirements within the device computation constraints and will also be good for the environment.

A smaller model will be simpler to train, but it may not be as accurate as we want. It is also important to note that AI applications should be accessible on edge devices without the need of external servers. Applications like those including face recognition need more accurate identifications because this might be used for serious issues like device security. Remote servers can be used for this problem, but that would require the device to send the data to the remote server and the server need to send information back to the device, which would make the whole process really slow. This might be very inconvenient for users who would then have to wait for simple tasks like unlocking their mobile devices. [4] Thus we may need to look at other methods.

II. RELATED WORKS

Keeping in mind all these constraints, model compression techniques can be used. One of those technique is parameter pruning which can help achieve a sweet spot between accuracy and efficiency. Parameter pruning involves removing some weights thus reducing model size. Knowledge distillation is another option for creating efficient models. In the knowledge distillation paradigm, a large complex pretrained model called teacher model is taken and finetuned to suit our task. That big model is then used to train a small model and when the student model is finally taken as the model for prediction. The smaller model learns from the larger model that is maintaining accuracy on a smaller efficient model.

[2] proposes an Ensemble Knowledge Distillation (EKD) framework to improve model generalization and efficiency in low-resource settings. The method uses a small CNN model with parallel branches that learn from multiple big teacher models. Thus they can capture all the learning from the large teacher models, but are also small for more efficient forward pass for inference. By ensemble learning from multiple teacher models, the student model becomes more accurate and they showed this with experiments.

[5] introduces a method that applies knowledge distillation to iterative generative models like diffusion and score-based models. It compresses their multi-step denoising process into a single-step "Denoising Student," achieving sampling speeds comparable to GANs while maintaining high image quality on datasets such as CIFAR-10 and CelebA.

[1] notes that most knowledge distillation methods work well only when the original training data is available, which is not always possible due to privacy or security limits. Earlier studies tried generating synthetic samples or reconstructing data from the teacher model, but they often relied too much on one model and thus the data may not be consistent with reality or generalized enough. This situation made it impossible to produce quality results. To solve this, the authors propose a new framework that uses information from both the teacher and student models to generate useful samples and train a small, well-performing student model.

[3] highlights that most compression and knowledge distillation methods handle quantization, pruning, and distillation separately, which can lead to loss of accuracy on the model or unstable training. To overcome this, the authors introduce the Cluster-Quantized Knowledge Distillation (CQKD) framework, which integrates cluster-based weight quantization directly into the training loop of both teacher and student models. This joint approach reduces parameters significantly while maintaining performance, providing an efficient and stable solution for deploying deep neural networks in resource-limited environments.

III. SOLUTION

A. Description of the Dataset

The CIFAR-10 dataset is one of the most classic popular datasets used to test image classification models for baseline models. It has 60,000 small color images, each 32×32 pixels

Here are the classes in the dataset, as well as 10 random images from each:

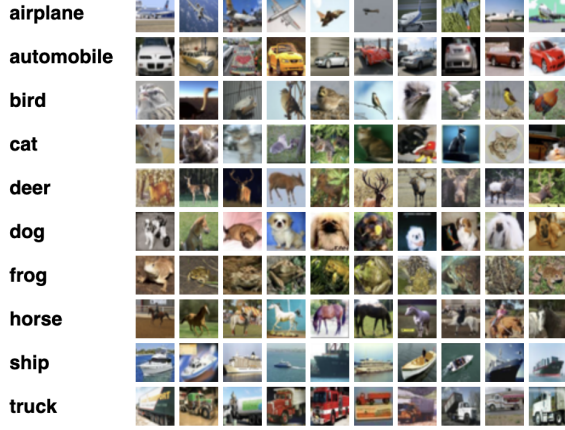


Fig. 1. CIFAR 10 Dataset, <https://www.cs.toronto.edu/~kriz/cifar.html>

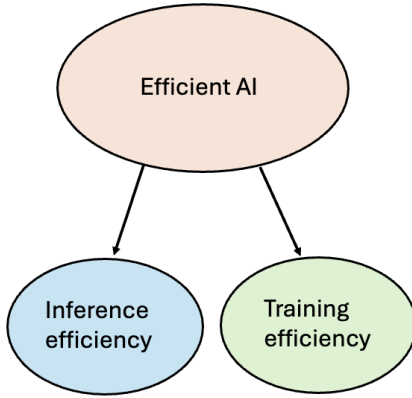


Fig. 2. Types of Efficiency

in size, grouped into 10 categories that are airplanes, cars, birds, cats, trucks, horses, ships, cars, dogs, frog and deer. Out of these, 50,000 images are used for training the machine learning model and 10,000 for testing accuracy of the model on unseen data. Even though the images are small, CIFAR-10 is still challenging and is often used to check how well deep learning and model compression techniques work on limited data. [6]

B. Machine Learning Algorithms

In this project, we use a teacher–student setup based on Knowledge Distillation (KD) to create image classification



Fig. 3. Forward Pass efficiency

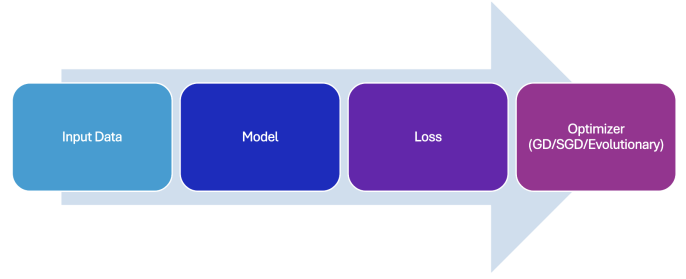


Fig. 4. Training efficiency

models that are both accurate and efficient. We use two convolutional neural networks (CNNs) that complement each other: ResNet-50 as the teacher and our own StudentCIFAR10Net as the student.

ResNet-50 [7] is a deep network with residual connections that help gradients flow through many layers. This lets the model learn complex features and achieve high accuracy, but it is computationally heavy.

With Knowledge Distillation, the student learns from both the true labels and the teacher’s soft predictions instead of the hard predictions. This technique was introduced by Hinton. This “dark knowledge” helps the student mimic the teacher’s decision patterns, improving generalization while keeping the model small and fast. Together, these algorithms let us balance performance and efficiency, which is important for real-world applications with limited computing resources. In some part of the worlds like Africa where internet access is limited, having models that run locally, fast on local device is very important.

Algorithm 1 Simulated Annealing (SA)

Require: Initial solution S , initial temperature T_0 , cooling rate α , objective function $E(S)$

Ensure: Best solution S^*

```

1:  $S^* \leftarrow S$ 
2: while stopping criterion not met do
3:   Generate a new candidate solution  $S'$  in the neighborhood of  $S$ 
4:   Compute energy difference  $\Delta E \leftarrow E(S') - E(S)$ 
5:   if  $\Delta E < 0$  then
6:     New solution:  $S \leftarrow S'$ 
7:   else
8:     Accept with probability  $P \leftarrow e^{-\Delta E/T}$ 
9:     Draw random number  $r \in [0, 1]$ 
10:    if  $r < P$  then
11:       $S \leftarrow S'$ 
12:    end if
13:  end if
14:  Update temperature:  $T \leftarrow \alpha \cdot T$ 
15:  if  $E(S) < E(S^*)$  then
16:     $S^* \leftarrow S$ 
17:  end if
18: end while
19: return  $S^*$ 
  
```

We also looked at non gradient based nature inspired meta-heuristic optimizers like Simulated Annealing and Genetic Algorithm for model training. These algorithms are inspired by nature and does not compute gradient. Genetic Algorithm is a well known and promising optimizer in the field. We chose simulated annealing as well because it is designed to balance exploration and exploitation. One problem with current gradient based optimizers is that they can get stuck in local optima. They are only designed for exploitation and not much exploration. Thus, algorithms like SA and GA can be promising. One drawback is that nature inspired algorithms depend on initial point a lot. Initial choice of parameters does not depend much for gradient based optimizers, but they do matter here.

C. Implementation Details

To evaluate the efficacy of knowledge distillation across disparate architectures, we implemented the teacher student network in Pytorch. The experiments were conducted on the CIFAR-10 dataset, which consists of 60,000 32×32 color images across 10 classes. Data preprocessing included normalization.

1) *Model Architectures*: The distillation framework bridges the gap between a high-capacity residual network and a lightweight standard convolutional network:

- **Teacher Model (ResNet-50)**: A deep residual network fine-tuned from ImageNet pre-trained weights. To preserve learned feature representations, the backbone layers were frozen, and only the final fully connected classification head was fine-tuned on CIFAR-10 for 10 epochs.
- **Student Model (StudentCIFAR10Net)**: We designed a custom CNN model to serve as the student. Unlike the teacher, this architecture does not use residual skip connections but has a straightforward stack of convolutional layers and pooling operations. This architectural constraint tests the ability of KD to transfer knowledge from a deep feature space to a shallower, simple small model.

Training was executed on Google Colab, but using the CPU runtime for all experiments. Both models were optimized using the Adam optimizer with a learning rate of 1e-3.

2) *Distillation Objective*: The student model is trained to minimize a weighted hybrid loss function that combines the standard Cross-Entropy (CE) loss with the Kullback-Leibler (KL) Divergence between the student and teacher logits. The total loss $\mathcal{L}_{KD}$ is defined as:

$$\mathcal{L}_{KD} = \alpha T^2 \cdot \mathcal{D}_{KL}(\sigma(\mathbf{z}_s/T) \parallel \sigma(\mathbf{z}_t/T)) + (1-\alpha) \cdot \mathcal{L}_{CE}(\hat{y}_s, y) \quad (1)$$

where $\mathbf{z}_s$ and $\mathbf{z}_t$ are the logits of the student and teacher respectively, $\sigma(\cdot)$ is the softmax function, y is the ground truth label, α is the balancing coefficient, and T is the temperature parameter that softens the probability distributions to reveal inter-class relationships (dark knowledge).

3) *Automated Hyperparameter Tuning via RL*: Finding the optimal balance between the teacher's guidance (T, α) and the

ground truth is non-trivial. To address this, we implemented an Epsilon-Greedy Multi-Armed Bandit algorithm [8]. We treat the selection of hyperparameters (T, α) as a discrete action space. The agent selects a configuration, trains a student model to convergence, and receives the validation accuracy as the reward signal.

Unlike static grid searches, this method dynamically balances exploration (testing new parameters) and exploitation (refining the best-known configuration). The algorithm incorporates an ε -decay mechanism to ensure convergence towards the optimal policy as the number of episodes increases.

Algorithm 2 Bandit-Based Hyperparameter Optimization for Knowledge Distillation

Require: Teacher model T , dataset $\mathcal{D}$, hyperparameter space

$\mathcal{A} = \{(T_i, \alpha_i)\}_{i=1}^N$, episodes E , exploration rate ε

Ensure: Optimal hyperparameters (T^*, α^*)

- 1: Initialize bandit agent B with N actions and exploration rate ε
 - 2: $A^* \leftarrow 0$
 - 3: **for** $e = 1$ to E **do**
 - 4: $(T, \alpha) \leftarrow B.\text{select_action}()$ $\triangleright \varepsilon$ -greedy selection
 - 5: Train student S with temperature T and distillation weight α
 - 6: $A \leftarrow \text{evaluate}(S, \mathcal{D})$ $\triangleright$ Accuracy as reward
 - 7: $B.\text{update}((T, \alpha), A)$ $\triangleright$ Update value estimates
 - 8: **if** $A > A^*$ **then**
 - 9: $A^* \leftarrow A, (T^*, \alpha^*) \leftarrow (T, \alpha)$
 - 10: **end if**
 - 11: **end for**
 - 12: **return** (T^*, α^*)
-

D. Novel Solution

By efficient, we are looking for runtime only for this work.

a) *Approach 1*: Our work explores the use of reinforcement learning (RL) to automate hyperparameter tuning in knowledge distillation (KD). Instead of manually setting parameters like temperature (T) and blending coefficient (α), we use an epsilon-greedy multi-armed bandit algorithm [8] to adjust them dynamically during training. Hyperparameter tuning for the knowledge distillation is automatically done by the Reinforcement Learning algorithm. This approach aims to make the Knowledge Distillation process more adaptive and reduce manual tuning. This may reduce the inference time in the student model, and that is what have been explored in this work.

b) *Approach 2*: We also explored pruning technique, this is for baseline comparison as pruning is the most straightforward approach. This is our baseline instead of our custom vanilla CNN as we are comparing efficient AI methods here, accuracy of pure CNN will intuitively be more. The pruned model is investigated to whether have faster inference time with not much degraded accuracy.

c) *Approach 3*: Our hypothesis is that the root cause of 'inefficiency' in deep models mainly come from gradient

calculations and big matrix multiplications during forward and backpropagation. So what if we use non gradient based optimization methods? They might not be stuck in a bad local minima, which is a problem with gradient descent as loss landscape of NN is non convex. This may not be generalized approach as hyperparameter tuning in EAs is complex, but while using CIFAR10 dataset and using a light CNN model, this might just be more efficient than gradient based optimizers. After reading through EA methods, Simulated Annealing [9] and Genetic Algorithm [10] seems to be the ideal candidate for us. It is simple, inspired by nature and the algorithm is more equipped for exploration vs exploitation than gradient based optimizers.

IV. RESULTS

TABLE I
CIFAR10 DATA ON CPU USING COLAB

Method	Accuracy (%)	Training Time (sec)	Inference Time
Vanilla CNN	74.58	951.82	10.28
Pruned Model	73.71	951.82	10.5
KD Model student *	54.87	822.62	5.79
KD Model teacher *	49.45	936.29	77.25
KD + RL	59.05	777.47	5.4
SA	9.31	542.21	10.84
GA	11.64	263.06	6.96

* model trained for 2 epochs

All model inference time shown are for 10000 datapoints

We ran all our models in google colab cpu. This is a choice we deliberately made as GPUs are not always accessible, thus we wanted to test if our algorithms are good even for basic CPUs.

a) *Our vanilla CNN model:* (CIFAR10Net) is a simple custom made convolution network for experiments in this project. The input image has RGB channels and is 32x32x3 and is passed through three convolution layers and then flattened and passed through a fully connected layer and an output layer to get the output of 10x1. We ran it for 10 epochs and tested it using 10000 datapoints. The inference time is around 10.28 seconds for 10k images and training time is 951.82 seconds with accuracy of 74.58 using Adam optimizer.

b) *The pruned model:* for this we used the CIFAR10Net model which is custom made model we built ourselves and after training the model for 10 epochs and using the Adam optimizer (we used same trained model from vanilla CNN), we first found out which of the parameters have value less than our threshold of 1e-3 and then pruned 20 percent of the less valued weights. The pruning ratio of 0.2 for all the layers is a hyperparameter, along with out threshold. The accuracy slightly decreased as seen from the table and no conclusion regarding inference time could be drawn.

c) *The Knowledge Distillation model:* we also used the RESNet50 as the teacher network and our own StudentCIFAR10Net as the student network. For the teacher network, we freeze the pretrained model and finetuned a fully connected layer on top of it to customize it for our CIFAR10 application. The teacher model was trained for only 2 epochs as the training

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 32, 32]	896
MaxPool2d-2	[-1, 32, 16, 16]	0
Conv2d-3	[-1, 64, 16, 16]	18,496
MaxPool2d-4	[-1, 64, 8, 8]	0
Conv2d-5	[-1, 64, 8, 8]	36,928
MaxPool2d-6	[-1, 64, 4, 4]	0
Linear-7	[-1, 128]	131,200
Dropout-8	[-1, 128]	0
Linear-9	[-1, 10]	1,290
=====		
Total params: 188,810		
Trainable params: 188,810		
Non-trainable params: 0		
=====		
Input size (MB): 0.01		
Forward/backward pass size (MB): 0.51		
Params size (MB): 0.72		
Estimated Total Size (MB): 1.24		
=====		

Fig. 5. Architecture Summary of CIFAR10Net

time was too high using the CPU (936.29 seconds for 2 epochs). The student model was also trained for 2 epochs taking 822.62 seconds. The student model had significantly less inference time compared to the pruned model and vanilla CNN (5.79 seconds), but accuracy was not good. This is probably because the model did not converge as we ran it only for 2 epochs and if we had ran it for more epochs the accuracy would have increased. This was a conscious choice from our side as we tried to keep training time as one of our priorities as well. The KD model is taking approximately 5 times more training time than vanilla CNN, thus even with a faster inference time, this did not look like a feasible solution for us.

d) *KD with hyperparameter tuning using Reinforcement learning methods:* We employed an Epsilon-Greedy Multi-Armed Bandit (MAB) algorithm to optimize the hyperparameters of the Knowledge Distillation (KD) process, specifically the Temperature and the Alpha mixing parameter. The MAB will act as a hyperparameter autotune method. The algorithm explored a discrete set of hyperparameter actions, using the resulting student model's test accuracy as the reward for the chosen action. The agent utilized an exploration probability $\epsilon = 0.2$. After running the auto-tuning for 2 episodes (with the student trained for 2 epochs per episode), the best hyper parameters were identified as Temperature=1 and Alpha=0.3. After the best parameters were chosen, the student model was then retrained using these optimized hyper parameters. Using optimized hyperparameters $\rightarrow$ Temperature=5, Alpha=0.3

- Epoch [1/2] - Loss: 1.2386 Time: 389.24s
- Epoch [2/2] - Loss: 1.0720 Time: 388.03s

Total Training Time: 777.47 seconds Test Accuracy: 59.05% Runtime (Inference): 5.4 seconds

The optimization process resulted in a test accuracy of 59.05%. However, the total training time of 777.47 seconds remains high, comparable to the teacher network training. The inference time of 5.4 seconds is a significantly lower time.

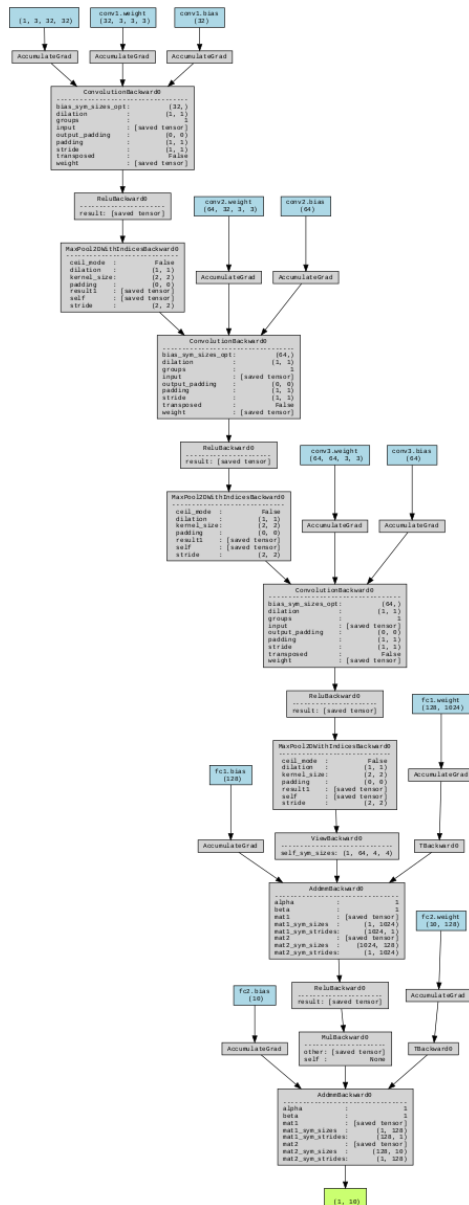


Fig. 6. CIFAR10Net

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 32, 32]	448
MaxPool2d-2	[-1, 16, 16, 16]	0
Linear-3	[-1, 10]	40,970
Total params: 41,418		
Trainable params: 41,418		
Non-trainable params: 0		
Input size (MB): 0.01		
Forward/backward pass size (MB): 0.16		
Params size (MB): 0.16		
Estimated Total Size (MB): 0.33		

Fig. 7. Architecture Summary of StudentCIFAR10Net

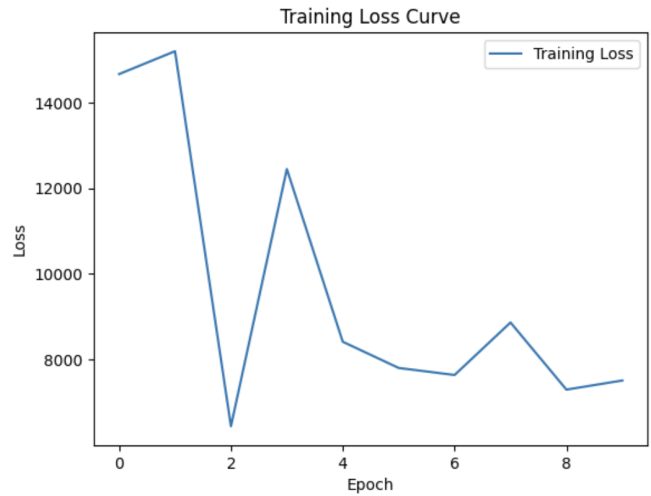


Fig. 8. Loss curve with Simulated Annealing

e) Training using Simulated Annealing: we used nevergrad package for implementing the simulated annealing algorithm. We trained the vanilla CNN using SA as optimizer and ran the optimizer for 10 epochs. It was surprisingly much faster (542.21 seconds) than the gradient based optimizer (951.82 seconds), but accuracy is very low around 10 percent which implies the optimizer did not reach any local or global minima in 10 epochs. But the inference time (10.84 seconds) for 10k images is at par with gradient based methods.

f) *Training using Genetic Algorithm:* we used the torchGA library from the pygad package for implementing this. Since this is a very slow optimizer, we even experimented with using subsets of the datapoints, but that did not help in model training. The training time was also surprisingly fast for 10 epochs (263.06 seconds), but the model still had low accuracy around 11 percent indicating the model did not learn anything. The inference time is surprisingly faster (6.96 seconds) for 10k datapoints, which is significant and can imply that if we can get a good enough optima from the training, this is the best algorithm for inference.

V. DISCUSSION

Our investigation in efficient Artificial Intelligence is two fold. Due to time and resource constraints, we have experimented with 10 epochs using only the CPU on Google Colab. Here we investigated the cases where we train the model and when we will use it for predcition, i.e., the forward pass, we would want it to be faster. For that we found that pruning is a very simple and effective method for that task, that's why we used it as baseline to compare our solutions against it. Our comparison baseline is Adam, so we only ran the other optimizers till their training time is less than or equal to Adam. One of our findings is that, provided we can find a good enough optima from nature inspired algorithms, we can see a significant improvement in inference time specially using genetic algorithm. Increasing the epochs and using GPU might as well help.

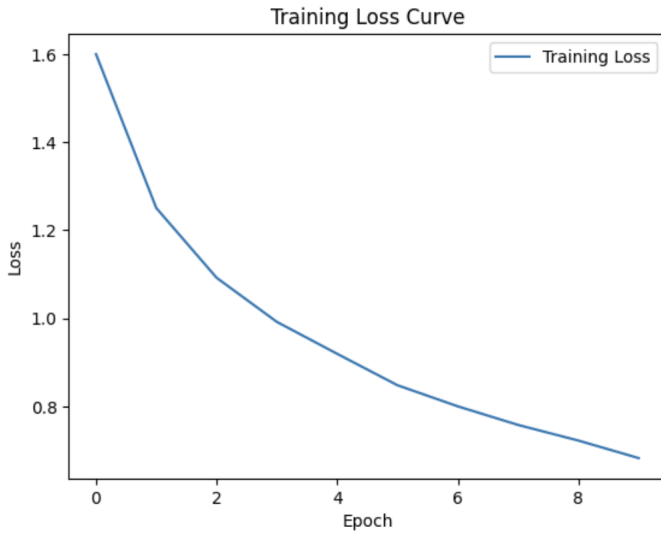


Fig. 9. Loss curve with Adam

VI. LIMITATIONS AND FUTURE WORK

This work has several limitations.

Our resources were limited so we could not run our experiments on GPUs or for more number of epochs. Though we plotted loss curves to make sure with 10 epochs the loss is not too bad, but more epochs could have been better. This was one reason we could not use stochastic gradient descent as a baseline optimizer, as it required more epochs for convergence according to our experiment, and we wanted same number of epochs for most of the experiments.

More hyperparameter tuning could have given us more clearer pictures of utility of the algorithms, especially for the nature inspired optimizers.

One future scope we would want to explore is Quantum Machine Learning algorithms as more efficient algorithms. Some promises of the quantum computing paradigm are exponential speedup and finding global optima in non convex loss scapes. Though full powered quantum machine learning is still distant, hybrid variational quantum circuits are feasible now. In case of hybrid QML, the forward pass calculation is done on a quantum circuit and the optimization and parameter update is done in a classical computer. Hybrid QML algorithms in some cases require less number of trainable parameters, which might indicate faster training and inference times. For example, [11] showed QML model can achieve high accuracy for MNIST dataset with 8 times less parameters than classical models. We can also explore other optimizers under the quantum computing paradigm like quantum annealing.

We can also try some more metaheuristic algorithms as our classical optimizers, combine new optimizers with pruning and knowledge distillation.

In the future we will also try to look into memory requirements, be it RAM or external memory. Since the nature inspired meta heuristic algorithms do not calculate gradient information, they may require less memory for storage of

gradient information, thus they can make the model more memory efficient.

In conclusion, we understood that the importance of efficient AI through this work we have done. We only explored CIFAR10 dataset application, but for a stronger conclusions of our hypothesis, we need to test our models on more variety of datasets and also some theoretical proofs if possible. Efficiency can mean many things, efficient inference time, efficient training time, efficient both, efficient memory, also particularly, efficiency for a particular task or generalized efficiency. For example, if we only need efficient algorithm for a task of robot grasping a particular object, does not matter if the algorithm is efficient for other tasks, that can also be a case. Or, we might also want an efficient algorithm for all tasks in the field. Thus, for our future work on efficiency, first we will understand the problem and the kind of efficiency required and will work on them.

REFERENCES

- [1] H. Zhao, X. Sun, J. Dong, M. Manic, H. Zhou, and H. Yu, "Dual discriminator adversarial distillation for data-free model compression," *arXiv preprint arXiv:2101.02388*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.02388>
- [2] U. Asif, J. Tang, and S. Harrer, "Ensemble knowledge distillation for learning improved and efficient networks," *arXiv preprint arXiv:1909.08097*, 2019. [Online]. Available: <https://arxiv.org/abs/1909.08097>
- [3] N. A. Khan and A. M. S. Rafat, "Optimizing deep learning models for resource-constrained environments with cluster-quantized knowledge distillation," *arXiv preprint arXiv:2305.08075*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.08075>
- [4] I. Khan and M. Mandal, "Analyzing compression techniques for computer vision," *arXiv preprint arXiv:2301.XXXXX*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.XXXXX>
- [5] E. Luhman and T. Luhman, "Knowledge distillation in iterative generative models for improved sampling speed," *arXiv preprint arXiv:2101.02384*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.02384>
- [6] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009, technical Report. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT press, 2018.
- [9] D. Bertsimas and J. Tsitsiklis, "Simulated annealing," *Statistical science*, vol. 8, no. 1, pp. 10–15, 1993.
- [10] T. V. Mathew, "Genetic algorithm," *Report submitted at IIT Bombay*, vol. 53, pp. 18–19, 2012.
- [11] A. Senokosov, A. Sedykh, A. Sagingalieva, B. Kyriacou, and A. Melnikov, "Quantum machine learning for image classification," *Machine Learning: Science and Technology*, vol. 5, no. 1, p. 015040, 2024.